method, you have to guard the comprehension with the `own` keyword (see chapter 4):

```
include = (klass, module) ->
  for own key, value of module
    klass[key] = value
```

Add the own keyword to filter out properties from the prototype chain.

This isn't bulletproof. To guarantee that you don't get properties you don't want, you must explicitly make your mixin an object that's removed from the prototype chain by making it have `null` as the prototype. This is done either by passing `Object.create` the value `null` as the object to create from

```
htmlRenderer = Object.create null
```

or by setting `null` as the prototype if you use a `Mixin` class:

```
class Mixin
  @:: = null
  constructor: (from, to) ->
    for key, val of from
      to[key] = val
```

The `null` value doesn't have any properties and it doesn't have a prototype. If you use it as a prototype, your object won't inherit any properties from the prototype chain.

5.8. Putting it together

A mixin is not only useful to solve Agtron's stock counting problem but more generally can be used to clean up some of the aspects of the client program, such as the view, that were embedded in the existing classes without belonging there. In the next listing you see a client program for Agtron's shop demonstrating the use of mixins to clean up the rendering code. Like other listings in this chapter, this is an example designed to run in a browser.

Listing 5.12. Agtron's shop client application